Add a section to your extension's manifest.json that describes where to find the plugin, along with other properties about it:

```
{
  "name": "My extension",
  ...
  "plugins": [
    { "path": "content_plugin.dll", "public": true },
    { "path": "extension_plugin.dll" }
  ],
  ...
   }
```

The "path" property specifies the path to your plugin, relative to the manifest file. The "public" property specifies whether your plugin can be accessed by regular web pages; the default is false, meaning only your extension can load the plugin.

Create an HTML file that loads your plugin by mime-type. Assuming your mime-type is "application/x-my-extension":

```
<embed type="application/x-my-extension" id="pluginId">
<script>
  var plugin = document.getElementById("pluginId");
  var result = plugin.myPluginMethod();  // call a method
in your plugin
  console.log("my plugin returned: " + result);
   </script>
```

This can be inside a background page or any other HTML page used by your extension. If your plugin is "public", you can even use a content script to programmatically insert your plugin into a web page.

Once you have an NPAPI plugin in your extension, the extension has unrestricted access to the local machine which is a security risk. If your plugin is not secure enough, a hacker could find loopholes and install potentially dangerous software on the user's computer. With so many security issues its best to avoid including an NPAPI plugin whenever possible.

Content Scripts

Javascripts are used to carry out various function that are needed to perform different actions on the page ranging from trivial operations like updating fields to accessing and painting some animation on the canvas. Scripts are thus a means by which we can change most of the content on the page